API Integration and Security Architecture

CSTAM 3.0 challenge "The Resilient IP Optimizer" (FelCloud) - Phase 2 deliverable
Repository: https://github.com/CSTAM-well-contained/monorepo

REST API reference (`cpd`)

Machine-readable spec: openapi.yaml (OpenAPI 3.1; a test fails if a route of the server is missing from it). Security model: security.md.

Base URL: `http://10.20.0.5:8443` on FelCloud (the only cp port the `cstam-cp` security group opens to the gateways), `http://10.20.0.1:8443` in the local demo.
All bodies are JSON. Authentication: `Authorization: Bearer <token>`.

- **admin token** (`CSTAM_ADMIN_TOKEN`): every `/v1` endpoint except the gateway ones;
- **gateway token** (`CSTAM_GATEWAY_TOKEN`): only `/v1/gateways/...`.

`GET /healthz` needs no token and answers `{"status":"ok"}`.

Read-only viewer token

`CSTAM_VIEWER_TOKEN` (optional, at least 16 characters, different from the other two) is accepted on every admin `GET` route and refused with **403** on every write. Give it to the jury: they can watch the dashboard live on their own laptop but cannot spin up, tear down, fail over or push a config. `GET /v1/whoami` answers `{"role": "admin"}` or `{"role": "viewer"}`.

Errors

`{"error": "<message>"}` with:

Code	When
400	invalid name, domain, port, image, JSON or parameter
401	missing or wrong token
404	unknown hackathon or team
415	body is not `application/json`
409	name already used, hackathon not active, lease already expired
503	IP pool exhausted (`Retry-After: 60`)
500	`internal error` (details only in the cpd log)

Hackathons

`POST /v1/hackathons` **-> 201**

```
{"name": "hack1", "domain": "hack1.cstam.felcloud.tn", "lease_ttl_seconds": 7200}
```

`name` is a DNS label; teams are published at `<team>.<domain>`. `lease_ttl_seconds` is the team lease (default 7200, at most 30 days).
Response: `{"id","name","domain","state":"active","lease_ttl_seconds","created_at"}`.

`GET /v1/hackathons` **-> 200,** `GET /v1/hackathons/{h}` **-> 200**

`DELETE /v1/hackathons/{h}` **-> 202**

Ends the hackathon: every team goes to `deleting`, routes are removed, sandboxes deleted, IPs quarantined then freed. The hackathon goes to `ended` (its name stays taken).

Teams

`POST /v1/hackathons/{h}/teams` **-> 202**

```
{"name": "team5", "app_port": 8080, "image": "nginx:1.27"}
```

`app_port` defaults to 80 and must be **80 or 8000-8099**, the ports the `cstam-team` security group opens to the gateways (400 otherwise). `image` is optional: without it the sandbox serves a page with the team
name and IP. The answer comes before the sandbox exists (`Location: /v1/teams/{id}`):

```
{"id": 12, "hackathon": "hack1", "name": "team5", "fqdn": "team5.hack1.cstam.felcloud.tn",
 "state": "provisioning", "ip": "10.20.0.36", "slot": 5, "app_port": 8080,
 "lease_expires_at": "2026-10-19T21:00:00Z", "created_at": "2026-10-19T19:00:00Z"}
```

States: `provisioning`, `ready` (routed), `failed` (`error` says why; cleaned up after 10 min), `deleting`, `deleted`.

`GET /v1/hackathons/{h}/teams` **-> 200,** `GET /v1/teams/{id}` **-> 200**

`POST /v1/teams/{id}/renew` **-> 200**

Sets the lease to now + the hackathon's `lease_ttl_seconds`: `{"lease_expires_at": "..."}`.
409 if the lease already expired or the team is not `provisioning` / `ready`.

`DELETE /v1/teams/{id}` **-> 202**

Starts the teardown (same order as a lease expiry).

Status

`GET /v1/status` **-> 200**

```
{"generation": 7, "config_version": "c4", "vip": "10.20.0.10",
 "ip_pool": {"allocated": 3, "free": 152, "quarantine": 4, "reserved": 1},
 "teams": {"ready": 3, "deleted": 4},
 "gateways": [{"name": "gw1", "acked_generation": 7, "config_version": "c4",
               "vrrp_state": "MASTER", "vrrp_since": "...", "last_seen": "...",
               "alive": true, "in_sync": true, "force_backup": false,
               "config_tried": "c4", "config_outcome": "applied"}],
 "alerts": []}
```

`alerts` is computed by cpd, never by a client: split brain (two MASTERs), no MASTER, a gateway silent for 40 s, behind the current generation, in FAULT, or refusing the newest config version.

Dashboard reads

Endpoint	Returns
`GET /v1/teams`	live teams of every hackathon, newest first
`GET /v1/events?after=ID&limit=N`	event log (team states, IP lease/quarantine/free, VRRP transitions, failovers, config outcomes), oldest first; without `after`, the last N
`GET /v1/ipam`	every address of the team pool with state, team and release time
`GET /v1/routes`	the routing table the gateways should serve now
`GET /v1/traffic?after=ID&host=FQDN&limit=N`	live traffic lines (client, host, method, path without query, status, timings, backend slot) and per-gateway req/s and 5xx over 10 s; in memory, best effort

Manual controls

Endpoint	Effect
`POST /v1/gateways/{gw}/failover[?force=true]` -> 202	planned failover: the gateway's agent writes 1 into keepalived's force-backup file, the instance goes to FAULT, the peer takes the VIP (about 0.3 s, notebook L10). 409 when no other gateway is alive, in sync, out of FAULT and not forced down, unless `force=true`
`POST /v1/gateways/{gw}/failback` -> 202	clears it; the gateway comes back as BACKUP (`nopreempt`)
`POST /v1/config` -> 201	new HAProxy config version `{"rate_limit": 1000, "extra": ["..."], "reason": "..."}`; each gateway checks, reloads hitlessly, verifies, rolls back by itself on failure, and reports `config_outcome`
`GET /v1/config` -> 200	versions, newest first
`POST /v1/config/{v}/rollback` -> 201	manual rollback: a new version with the content of `v` (history is never rewritten, so a gateway that was offline converges to the newest)

Gateway endpoints (used by `gwagent`)

`GET /v1/gateways/{gw}/desired?after=N&wait=25s` **-> 200**

Long-poll: answers at once if the routes generation differs from `N` (or without `after`), otherwise when it changes or after `wait` (at most 25 s).

```
{"generation": 7, "routes": [{"host": "team5.hack1.cstam.felcloud.tn", "slot": 5,
                              "addr": "10.20.0.36", "port": 8080}],
 "force_backup": [], "config": {"version": "c4", "rate_limit": 2000}}
```

`force_backup` and `config` are part of the same generation: a failover or a config push bumps it, which wakes the long-polls.

`POST /v1/gateways/{gw}/traffic` **-> 200**

Batches of at most 150 traffic lines, about once a second, plus the count of lines dropped on the gateway. Best effort: a failed batch is counted as dropped, never retried.

`POST /v1/gateways/{gw}/ack` **-> 200**

```
{"generation": 7, "config_version": "c4", "vrrp_state": "MASTER",
 "config_tried": "c4", "config_outcome": "applied", "config_error": ""}
```

Records what the gateway serves. A generation ahead of the database is refused (409): it would let the sweep free IPs too early.

Integrating a registration portal (client side)

A hackathon portal (or `curl`) drives the platform with the admin token:

1. `POST /v1/hackathons` once per event (`409` = it already exists: fine on a retry).
2. For each team: `POST /v1/hackathons/{h}/teams`. The answer (`202`) already holds the team's `ip`, `slot` and `fqdn`. **Retrying is safe**: a second POST with the same name gets `409`; then `GET /v1/hackathons/{h}/teams` finds the team. On `503` wait `Retry-After` seconds (pool full: IPs come back after teardown and the 60 s quarantine).
3. Poll `GET /v1/teams/{id}` (every 1-2 s) until `state` is `ready` (typically 2-5 s with the docker driver, about 25 s with one VM per team) or `failed` (`error` says why).
4. Call `POST /v1/teams/{id}/renew` before `lease_expires_at` to keep a team; otherwise it is torn down automatically.
5. `DELETE /v1/hackathons/{h}` at the end: every route is removed at once, sandboxes are deleted, IPs return to the pool when both gateways have acked.

Every answer is JSON; errors are `{"error": "..."}` with the codes above. Client errors (4xx) must not be retried unchanged; `503` and network errors may be.

OpenStack API integration (`cpd` -> FelCloud)

`cpd` is the only component with cloud rights (application credential from `clouds.yaml`, see security.md). The VM driver (`internal/openstack`, gophercloud v2) uses:

Step	API call	Notes
start	Keystone token from the application credential; Nova `GET /flavors`, Glance `GET /images`	flavor `G0.basic.1c1g` and image `Debian 12 - Bookworm` resolved once
create, 1	Neutron `GET /ports?name=cstam-<h>-<team>`	idempotency: an existing port with the same IP is reused
create, 2	Neutron `POST /ports` with `fixed_ips` = the team IP, `security_groups` = `cstam-team`	not retried (not idempotent); `409 IpAddressAlreadyAllocated` -> look again by name (our own lost answer) or `ErrIPTaken` (the IP stays quarantined)
create, 3	Neutron `PUT /ports/{id}/tags` (`cstam`, `cstam-team`)	for garbage collection by tag
create, 4	Nova `GET /servers?name=...`, then `POST /servers` with the port and cloud-init user-data	user-data serves the team page or runs the team image
create, 5	Nova `GET /servers/{id}` every 2 s until `ACTIVE` (`ERROR` -> team `failed`), 5 min max	
delete	Nova `DELETE /servers/{id}`, wait for `404`, then Neutron `DELETE /ports/{id}`	refuses names without `cstam-`; `404` counts as done
IP check	Neutron `GET /ports?network_id=...&fixed_ips=ip_address=...`	the sweep never frees an IP a port still holds
reconcile (every 2 s)	the create and delete calls above, driven by the database	resumes teams left in `provisioning` or `deleting` (e.g. after a cpd restart), cleans failed teams, expires leases. Resources are found by their deterministic name; there is no scan of the cloud for orphans (a row deleted by hand from the database would leave its VM: `List` by prefix and tag exists for that, used by the tests)

Errors: idempotent calls are retried 3 times (200, 400, 800 ms) on 429, 5xx and network errors; at most 8 calls in flight. Quota errors (`OverQuota`, 413) map to `ErrQuota` and the team fails with a clear message instead of retrying forever. Measured on FelCloud: VM ACTIVE in 23 s, server and port deleted in 14 s (benchmark B4). Static infrastructure (network, router, security groups, VIP and gateway ports, floating IPs) is created by Pulumi (`infra/pulumi`), not by `cpd`.

Example session

```
A=(-H "Authorization: Bearer $CSTAM_ADMIN_TOKEN" -H "Content-Type: application/json")
curl "${A[@]}" -X POST $CPD/v1/hackathons -d '{"name":"hack1","domain":"hack1.cstam.test"}'
curl "${A[@]}" -X POST $CPD/v1/hackathons/hack1/teams -d '{"name":"team1"}'
curl "${A[@]}" $CPD/v1/hackathons/hack1/teams          # wait for "state":"ready"
curl -H "Host: team1.hack1.cstam.test" http://10.20.0.10/
curl "${A[@]}" -X DELETE $CPD/v1/hackathons/hack1
```

Security architecture

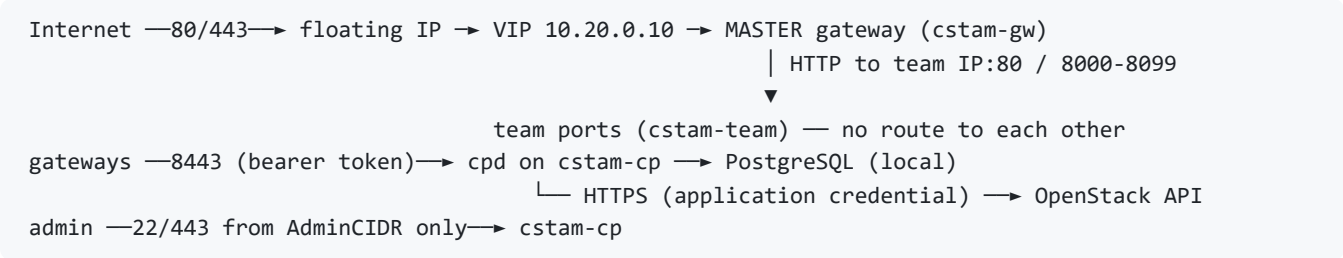
What the platform protects, from whom, how, and how each control is verified. Every control names the code or file that implements it and the test or measurement that checks it. Known gaps are listed at the end, not hidden.

1. Assets and threats

Asset	Why it matters
Team sandboxes and their traffic	a team must not read, change or reach another team's app
The gateways (HAProxy, keepalived, `gwagent`)	the single public entry; a takeover would redirect every team
The control plane (`cpd`, PostgreSQL)	decides which IP and which route each team gets
The OpenStack application credential	can create and delete VMs and ports in a project shared with others
Team IPs	a reused IP that still receives traffic for its old team leaks data to the new one

Actor	Can	Must not be able to
Internet visitor	send HTTP to the floating IP	reach anything but published team apps; see management endpoints; exhaust a gateway
Team (participant)	run any code in its own sandbox	reach other teams, the gateways' management, the control plane, the OpenStack API with our rights; spoof another IP
Compromised gateway	serve wrong routes	read the database, call OpenStack, free IPs early
Other tenants of the shared OpenStack project	see and change any project resource	(outside our control) we only ever modify resources named `cstam-*`

2. Trust boundaries



Only `cpd` holds the database DSN and the OpenStack credential. The agents hold a gateway token that can read routes and post acks, nothing else.

3. Network isolation (Neutron security groups)

Five role groups, 14 ingress rules, all in one table in `infra/pulumi/secgroups.go` (quota: 10 groups, 100 rules). Egress keeps the OpenStack default (allow).

Group (on)	Rule	From
`cstam-gw` (gw1, gw2, VIP port)	TCP 80, 443	`0.0.0.0/0` (public entry)
	TCP 22, TCP 8404 (stats/metrics), ICMP	`cstam-cp` only
`cstam-vrrp` (gw1, gw2)	IP protocol 112 (VRRP)	`cstam-vrrp` only: nobody else can send adverts
`cstam-cp` (cp)	TCP 22, 443	`AdminCIDR` only
	TCP 8443 (`cpd` , `config.CPDPort`)	`cstam-gw` , `cstam-host`
`cstam-host` (hackathon hosts, Phase 2)	TCP 22, ICMP	`cstam-cp` only
`cstam-team` (every team port)	TCP 80 and 8000-8099	`cstam-gw` only
`cstam-team` **egress**	any protocol to 0.0.0.0/0 **except 10.20.0.0/24** (24 prefixes, `infra/pulumi/egress.go`); no default rules	teams reach the internet and metadata, never cp, the gateways or each other

What is deliberately **absent**:

- No "self" rule on `cstam-team` : Neutron drops team-to-team traffic, even inside the subnet.
- No rule opens a port on an agent: agents dial out to `cpd` ; nothing can connect to them.
- `cstam-host` is never a remote group, so a team address added to a host can never inherit host rights.
- Team ports cannot reach `cpd` (8443 accepts only the gateway and host groups) or the gateways' SSH, stats or VRRP.

Anti-spoofing. Port security is on for every port: a team VM can only send from its own IP and MAC. Only the gateway ports carry the VIP in `allowed_address_pairs` , so no team can answer ARP for 10.20.0.10 or take the VIP.

App ports. `cpd` refuses a team whose `app_port` is not 80 or 8000-8099 (`config.TeamPorts` , `lifecycle.Spec.Validate`), so a route can never point at a port the firewall blocks.

4. The public frontend (HAProxy, `internal/gw/haproxy/haproxy.cfg.tmpl`)

Control	How
Management endpoints hidden	`/__gw/*` answers only to `InternalSources` (localhost, cp, both gateways); from anywhere else it is a plain 404 that names nothing
Stats and Prometheus metrics	separate frontend on 8404, reachable from `cstam-cp` only (security group)
Unknown hosts	404 page; there is no default backend that could leak a team
Per-team rate limit	stick table keyed on the normalized `Host` (lower case, port removed): 2000 requests per 10 s per Host by default (`-rate-limit` , per-Host overrides in `ratelimit.map`); above it `429` . A flooding client stays limited while it floods (refused requests count)
Slow clients	`timeout http-request 10s` , `client 30s` , `http-keep-alive 5s` , `queue 5s`
Team down	maintenance page with `Retry-After` , never another team's backend
Routing integrity	one backend per slot, one server per backend; the `hosts.map` swap is atomic; `gwagent` never applies an older generation (`TestNeverAppliesOlderGeneration`) and refuses invalid routes (`TestBadRouteIsNotApplied`)

Control	How
Config changes	`haproxy -c` before use, verification through the real frontend after reload, rollback on failure (benchmark B1)
Config pushed through the API (`POST /v1/config`)	admin token only; `extra` lines are a deliberate admin escape hatch placed inside the HTTP frontend: at most 20 lines of 1-200 printable ASCII characters, so no newline can open a new section (`TestConfigInValidate`); the agent validates the settings again and each gateway runs the same check, reload, verify, rollback (notebook L11). An admin can still write a rule that misroutes traffic: that is the admin's power, recorded in the event log (`config.created`, `config.rollback`)
Traffic log (`/v1/traffic`)	lines carry the client IP, Host, method and path **without the query string** (no tokens in URLs leak); read with the admin token only, kept in memory (2000 lines), never written to disk; every text field is cut to a fixed size on the agent and again in cpd, a batch is at most 150 lines and 512 KiB (`TestClip`, `TestTrafficEndpoints`); the gateway name comes from the URL, not from the line
Planned failover (`POST /v1/gateways/{gw}/failover`)	admin token only; refused (409) unless another gateway is alive, serving the current generation, out of FAULT and not forced down (`TestFailover`), so one click cannot take the public entry down

Measured (benchmark B8): one team flooded at ~79,000 requests/s for 20 s: 99.9 % answered 429, and the other 19 teams, at 2,850 requests/s together, had **0 errors** and a p99 latency of 3.7 ms.

5. Control-plane API (`internal/api`)

Control	How
Authentication	`Authorization: Bearer`; two tokens (admin, gateway), at least 16 characters, must differ (`Server.Validate`, `TestValidateTokens`); compared in constant time (`crypto/subtle`)
Authorization	each route declares its token (`routes()`); a gateway token cannot call admin routes and vice versa (`TestRoutes`); the OpenAPI spec is checked against the routes (`TestOpenAPIListsEveryRoute`)
Read-only role	optional viewer token: admin `GET` routes only, 403 on every write, 401 on gateway routes (`TestViewerToken`); for the jury's laptops during the pitch
Dashboard	served by cpd with a strict CSP (`script-src 'self'`, no framing), data inserted as text only, checked by `go test ./web/` (no `innerHTML` / `eval`, no inline script); see dashboard.md
Request limits	body at most 64 KiB, `application/json` only (415), unknown JSON fields rejected
Server timeouts	read header 5 s, read 15 s, write 40 s (long-poll 25 s + margin), idle 90 s
Errors	client errors say what is wrong; internal errors return `internal error` and are logged only on the server
Integrity of acks	an ack for a generation newer than the database is refused (409), so a buggy or hostile gateway cannot make the sweep free IPs early (`TestGatewayAckGatesRecycling`)
Secrets	tokens and DSN only from the environment (`/etc/cstam/*.env`, mode 0600), never from flags or files in the repo

6. Credentials and secrets

- **OpenStack**: a Keystone **application credential** (`auth_type: v3applicationcredential`), not a user password. Only `cpd` uses it (`OS_CLIENT_CONFIG_FILE=/etc/cstam/clouds.yaml`, 0600, owner `cstam`). The linter forbids gophercloud and pgx in agent code (`depguard` rule `agents-no-db-no-cloud`), so a gateway compromise exposes neither the cloud nor the database.

- **Repository**: `clouds.yaml`, `*.env`, keys and certificates are git-ignored; `gitleaks` scans the whole history in CI (`.github/workflows/ci.yml`), and in a local pre-commit hook on the machines where it is installed (hooks are not versioned).
- **Team VMs**: their cloud-init user-data contains no secret (it is readable from the metadata service by the VM itself): only the team name, hackathon, IP and port (`internal/openstack/userdata.go`).

7. Input validation and injection

Every value that reaches HAProxy, Docker, cloud-init or OpenStack is validated first:

- names (hackathon, team, gateway): `^[a-z0-9]([a-z0-9-]{0,30}[a-z0-9])?$`, which also keeps the resource name `cstam-<hackathon>-<team>` a valid DNS label;
- container image: strict reference pattern, at most 255 characters;
- team IP: must be inside the team pool (or the lab pool), never Neutron's own range;
- app port: the firewall ranges above.

`lifecycle.Spec.Validate` runs before both sandbox drivers and before the user-data template, so no value can break the YAML or a command (`TestSpecValidate`, `TestUserData`, `TestCreateRejectsInvalidSpec`). External commands run as argv lists, never through a shell (`gosec` in the linter).

8. Resource ownership and blast radius

- The OpenStack project is shared: the drivers only create resources named `cstam-*` and refuse to delete anything else (`Delete` checks the prefix; `TestDelete`). Team ports also carry the tags `cstam`, `cstam-team`.
- Pulumi manages only the platform it created; the VIP port and the public floating IP are `protect`ed. The OVN metadata port 10.20.0.200 is never touched.
- OpenStack calls are bounded (8 in flight) and retried only when idempotent (GET, DELETE, tag PUT); a lost port-create answer is detected by name, so a retry cannot create a second port.

9. IP reuse safety (reclamation)

An IP is freed only when (1) its route was removed in the same transaction that quarantined it, (2) every gateway seen in the last 5 minutes acked that generation, (3) 60 s passed and (4) Neutron shows no port holding it. So no gateway can still send a former team's traffic to the new owner of the address (`TestQuarantineSweep`, `TestGatewayAckGatesRecycling`, `TestIPTakenStaysQuarantined`; benchmark B7).

10. Host hardening (`deploy/systemd`)

Unit	Runs as	Hardening
`gwagent.service`	`cstam`, group `haproxy`	`NoNewPrivileges`, `ProtectSystem=strict` (writes only `/etc/haproxy`, `/var/lib/cstam`, `/run/cstam`, `/run/haproxy`), `ProtectHome`, `PrivateTmp`, empty `CapabilityBoundingSet`, `RestrictAddressFamilies`, native syscalls only
`cpd.service`	`cstam`	same, with `/etc/cstam` read-only

Unit	Runs as	Hardening
HAProxy	master root, workers `haproxy`	admin and master sockets `0660` group `haproxy`
`cstam-gw-bootstrap.service`	root, oneshot	only writes the HAProxy and keepalived files before they start

keepalived health and notify scripts run as `keepalived_script`, not root. Team containers in the local demo run with memory, CPU and pid limits, `no-new-privileges` and all capabilities dropped.

11. Verification summary

Claim	Evidence
Management endpoints invisible from outside	`TestGatewayGuards` (real HAProxy, outside source address)
Flood of one team limited, others unaffected	`TestGatewayGuards`; benchmark B8
Tokens separated and checked	`TestRoutes`, `TestValidateTokens`
No IP reused while a gateway may still route it	`TestGatewayAckGatesRecycling`, `TestQuarantineSweep`
No invalid name, image or port reaches the cloud	`TestSpecValidate`, `TestValidation`, `TestCreateRejectsInvalidSpec`
Only `cstam-*` resources deleted	`TestDelete`, `Driver.Delete` prefix check
Firewall table	`infra/pulumi/secgroups.go`, `egress.go`; **measured on FelCloud** with lab VMs (phase2.md, notebook L14): team to team, team to management, team to cp all dropped; gateway to team allowed

12. Known gaps and next steps

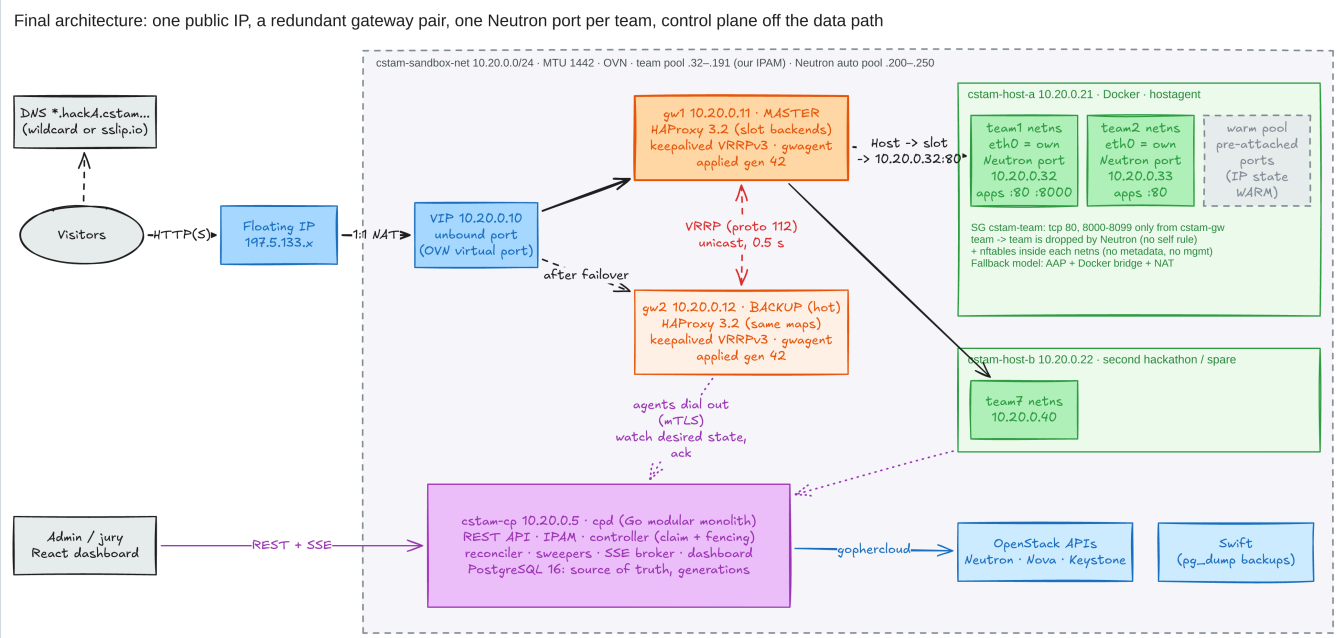
Gap	Risk	Plan
Agents talk to `cpd` in plain HTTP inside the private network	someone already on `cstam-sandbox-net` could read the gateway token	the security groups only let the gateway and host groups reach 8443; mTLS (`internal/pki`) is the Phase 3 step
One gateway token shared by gw1 and gw2	a compromised gateway could post acks as the other one	per-gateway tokens or mTLS client certificates
Rate-limit counters are per gateway (no `peers` section)	after a failover a flooding client starts from zero on the new MASTER	HAProxy `peers` between gw1 and gw2
~~Team egress is open~~ fixed	was: a sandbox could reach cp SSH (L14)	`cstam-team` egress only outside 10.20.0.0/24; the internet and metadata stay reachable
No TLS on the public entry yet	team sites are HTTP only	wildcard certificate (bonus, needs DNS from FelCloud)
The local Docker demo has no isolation between team containers	demo only	isolation is enforced by Neutron on FelCloud

Gap	Risk	Plan
gw1 and gw2 run on the same hypervisor (soft anti-affinity)	a hypervisor failure takes both	asked FelCloud; hard anti-affinity failed for lack of hosts

Architecture

1. Overview

One public floating IP, a redundant pair of gateways sharing a virtual IP, one private IP per team, and a control plane that is **not on the data path**: if `cpd` stops, routing and failover keep working with the last applied state.



The diagram shows the target design. What is built today (Phase 1) differs in three points:

- Team sandboxes are **one small VM per team** (`G0.basic.1c1g`) on a Neutron port carrying the team's IPAM address. The shared Docker hosts with per-team network namespaces (`hostagent`) are the Phase 2 design for more teams than the instance quota allows.
- Agents talk to `cpd` over **plain HTTP JSON with a bearer token** (long-poll + ack), not mTLS.
- No SSE dashboard yet (Phase 3).

Address (10.20.0.0/24, `cstam-sandbox-net`)	**Role**
.1	Neutron router (SNAT to the internet)
.5	`cstam-cp`: `cpd` + PostgreSQL
.10	VIP, an unbound Neutron port, held by the MASTER gateway; the floating IP maps to it
.11, .12	`cstam-gw1`, `cstam-gw2`
.32 - .191	team pool, allocated by our IPAM (never by Neutron)
.200 - .250	Neutron's own allocation pool (never used by us)

The plan lives in `internal/config/topology.go`, `infra/topology.env` and `infra/pulumi/config.go`. Static infrastructure (network, router, security groups, VIP port, gateways, control plane) is created by Pulumi (`infra/pulumi`, stack `prod`). Team resources are created at runtime by `cpd`.

2. Components

Component	Code	Job
`cpd`	`cmd/cpd`, `internal/api`, `internal/lifecycle`, `internal/ipam`, `internal/store`	REST API, team state machine, IPAM, reconciler. PostgreSQL is the source of truth.
Sandbox drivers	`internal/openstack` (gophercloud), `internal/docker` (docker CLI)	Create/delete the team sandbox at its fixed IP; idempotent by name `cstam-<hackathon>-<team>`
`gwagent`	`cmd/gwagent`, `internal/gw/agent`	On each gateway: long-polls the desired routes from `cpd`, applies them to HAProxy, acks the generation and reports the VRRP state
Applier	`internal/gw/applier`, `internal/gw/haproxy`	Renders `haproxy.cfg` and map files, changes routes at runtime, reloads safely, rolls back
keepalived config	`internal/gw/keepalived`	VRRPv3 unicast between gw1 (priority 150) and gw2 (100), `nopreempt`, health script, notify script
systemd units	`deploy/systemd`	bootstrap (root) writes the HAProxy and keepalived files; HAProxy and keepalived from the packages; `gwagent` unprivileged (`cstam`, group `haproxy`); `cpd`

3. HA gateway data flow

```
visitor --HTTP--> floating IP --1:1 NAT (OVN)--> VIP 10.20.0.10 --> MASTER gateway
   MASTER HAProxy frontend:
     Host header -> hosts.map -> be_slotNNN -> server = team IP:app_port   (data path)
   gwagent <--long-poll desired / POST ack--> cpd                          (control path)
   keepalived <--VRRP adverts every 0.5 s--> peer keepalived              (failover)
```

Routing. HAProxy has 128 backends `be_slot001..128`, each with one server. Publishing a team sets that slot's server address with the runtime API and swaps the `hosts.map` atomically (a versioned map swap). No reload is needed to add or remove a team: a new team is routed 24 ms after `cpd` publishes it (benchmark B2). Unknown hosts get a 404 page, a team whose sandbox is down gets a maintenance page, and each `Host` is rate limited.

Desired state and acks. Every change of the published routes increases a global `routes_generation` in PostgreSQL. A gateway always applies the whole table of one generation, never a diff, and never an older generation than the one it serves. It acks only a generation it has applied, and it saves the table to `/var/lib/cstam/desired.json` so a restart serves the same routes even without `cpd`.

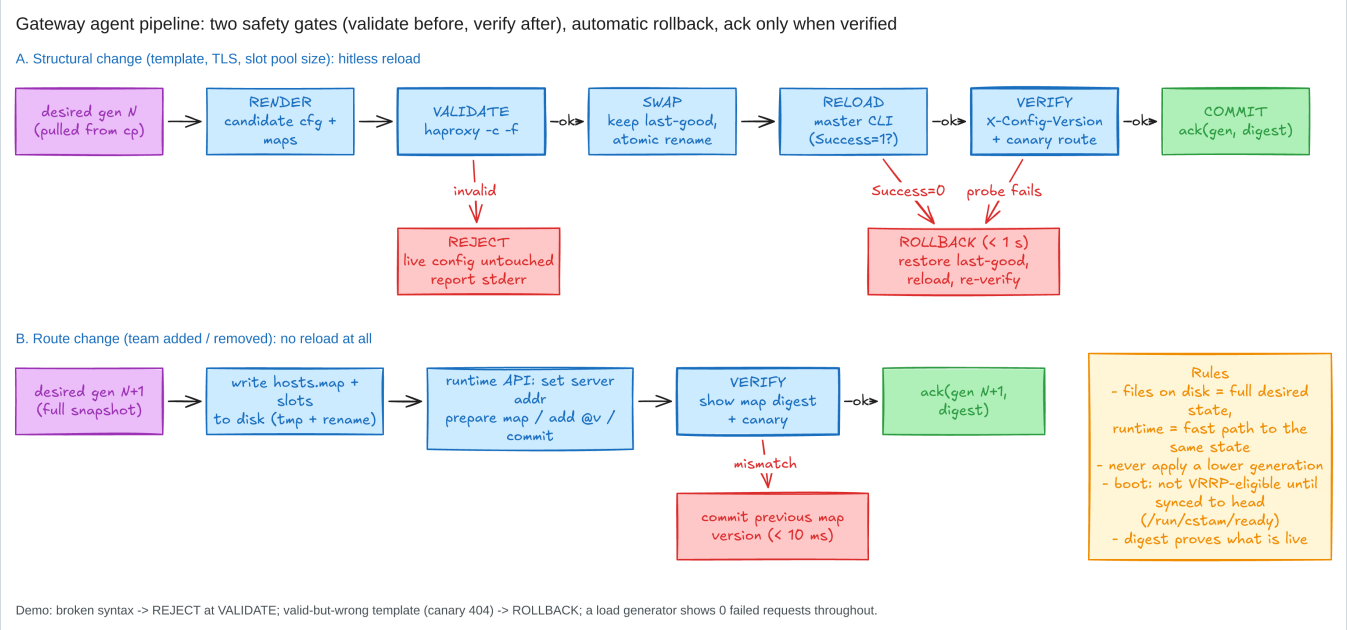
Readiness. At start `gwagent` deletes `/run/cstam/ready` and recreates it only after a successful sync with `cpd` (or after 10 s without `cpd`, as `stale`). keepalived's `chk_gateway` needs that file **and** a 200 from HAProxy's own `/__gw/healthz`, so a freshly started gateway does not take the VIP before it has routes. If `cpd` is down, the gateway becomes ready after 10 s with the routes it saved before (better than no gateway at all); it catches up when `cpd` returns.

Failover (diagram):

- MASTER silent (crash, freeze): the backup declares it dead after 3 x 0.5 s + skew, takes the VIP and sends gratuitous ARPs; OVN moves the virtual port binding and the floating IP follows.

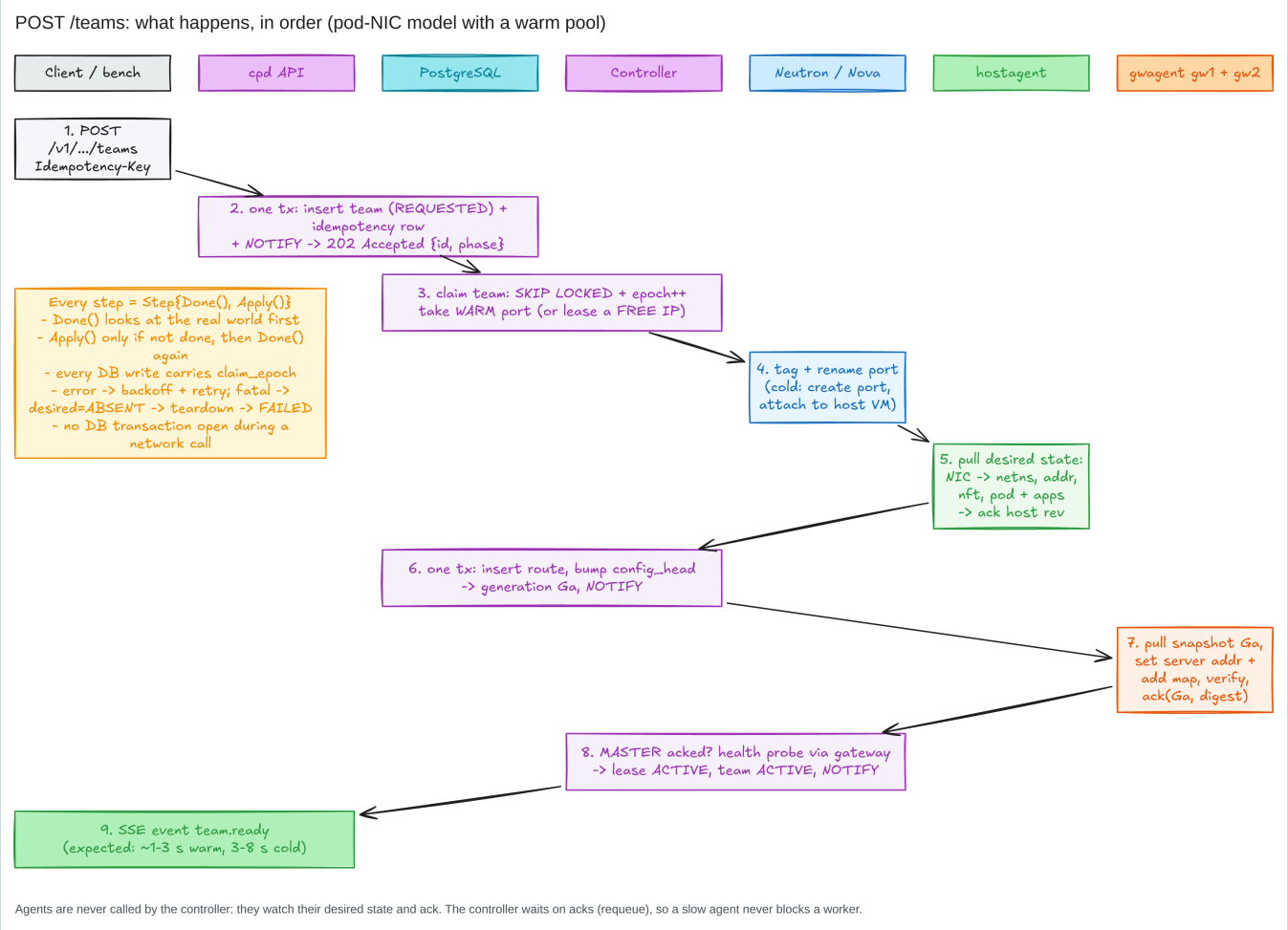
- HAProxy dies: systemd restarts it at once (about 0.5 s of downtime, no failover). If it stays down or stops answering, `chk_gateway` fails twice (interval 1 s), the gateway goes FAULT and gives up the VIP. On the VMs `vrrp_track_process` also reacts at once when the process dies.
- Manual failover: write `1` to `/run/cstam/force-backup`.
- `nopreempt`: a gateway that comes back stays BACKUP, so there is no second switch.
- Open TCP connections on the failed gateway are lost (HAProxy is a full proxy); every new connection succeeds after the gap.

4. Safe reload and rollback



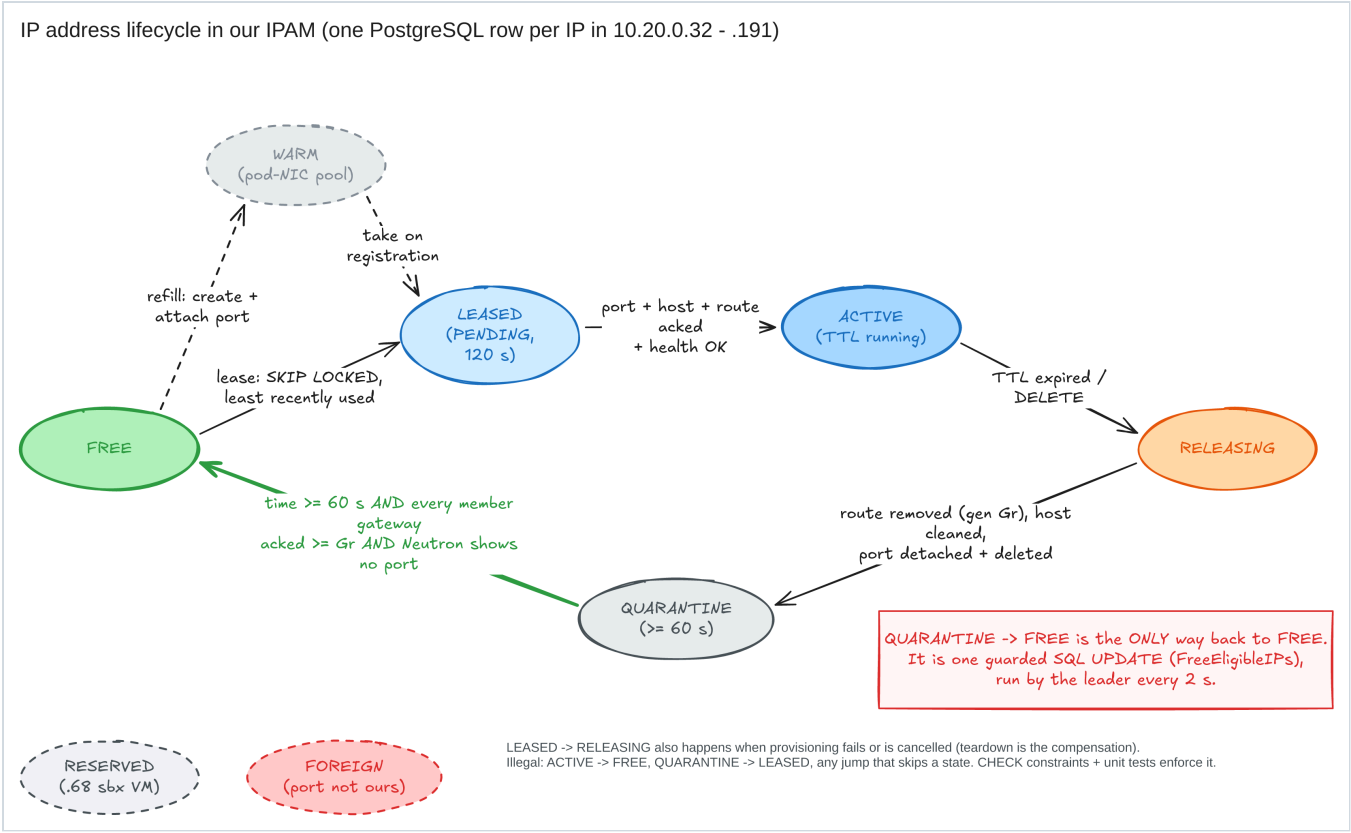
Config changes (not route changes) go through the applier: render -> `haproxy -c` (rejected configs never reach HAProxy) -> atomic file swap -> `reload` on the master CLI (HAProxy 3.2 answers `Success=1` only when the new workers loaded) -> verify through the real frontend (`/__gw/healthz` must show the new `X-Config-Version`, and the canary backend must answer) -> if anything fails, restore the last good config and reload again. Old workers finish their connections, so a reload drops no request (benchmark B1).

5. Team lifecycle and IPAM



`POST /v1/hackathons/{h}/teams` inserts the team and allocates its IP and slot **in the same transaction** (lowest free IP, lowest free slot, under an advisory lock so concurrent requests never get the same one), then answers `202` with the team in state `provisioning`. A worker (`cpd -workers`, default 4) creates the sandbox at that IP, then marks the team `ready` and bumps the routes generation: both gateways route it within milliseconds.

States: `provisioning -> ready | failed -> deleting -> deleted`. A failed team keeps its name and slot for 10 minutes so the error can be read, then it is cleaned up.



Reclamation (diagram). A team is deleted when its lease expires (`lease_ttl_seconds` of the hackathon), when an admin deletes it, or when its hackathon ends (`POST /v1/teams/{id}/renew` resets the lease to now + TTL). Order:

1. the route is removed and the routes generation bumped, in the same transaction that puts the IP in `quarantine` with `released_gen` = that generation;
2. the sandbox is deleted;
3. the sweep (every 2 s) frees the IP only when it has been quarantined for 60 s, **every gateway seen in the last 5 minutes acked a generation >=** `released_gen`, and the driver says no resource still holds the IP (Neutron port check).

This is what makes aggressive recycling safe: no gateway can still send traffic for the old team to an address that a new team now uses. Every IP change writes an `events` row.

6. Security

Summary below; the full threat model, firewall table, verification and known gaps are in security.md.

- **Isolation between teams**: the `cstam-team` security group only accepts TCP 80 and 8000-8099 from the `cstam-gw` group, and has no "self" rule, so Neutron drops team-to-team traffic. Teams cannot reach each other, the gateways' management, or the control plane.
  (The local Docker demo does not reproduce this: all containers share one bridge.)
- **Gateways**: VRRP (IP protocol 112) only between gateway ports (`cstam-vrrp` group); `/__gw/*` endpoints answer only to internal sources (localhost, cp, gateways).
- **Control plane**: admin and gateway tokens are different, compared in constant time, and only read from the environment. Internal errors are never returned to clients. The OpenStack

credential is an application credential, used only by `cpd`; agents never import gophercloud or pgx (enforced by the linter).

- **Input validation**: every name that reaches HAProxy, Docker or OpenStack must match `^[a-z0-9]([a-z0-9-]{0,30}[a-z0-9])?$`; images are checked against a strict reference pattern; external commands run with argv lists, never through a shell.
- **Resource ownership**: we only create, touch and delete resources named `cstam-*` (the OpenStack project is shared). The sandbox drivers refuse to delete anything else.
- **Team containers (docker driver)**: memory, CPU and pids limits, `no-new-privileges`, all capabilities dropped for the default page.